

PostgreSQL Cleanup

Reset Primary & Standby to Pre-Restore State

Item	Detail
PostgreSQL Version	16
Replication Slot Name	ssncpri
Data Directory	/apps/pgsql_data/16
Goal	Clean up everything so both servers are ready for next week's restore attempt

OBJECTIVE: Undo the failed restore, remove the partially loaded data, revert all parameter changes, rebuild streaming replication between primary and standby, and leave both servers in a clean, healthy state — exactly as they were before the restore was attempted.

Execution Order Summary

Step	Action	Server
1	Stop the standby	STANDBY
2	Cancel/stop the active restore on primary	PRIMARY
3	Drop the restored database (remove partial data)	PRIMARY
4	Drop the broken replication slot ssncpri	PRIMARY
5	Revert all ALTER SYSTEM parameter overrides	PRIMARY
6	Restart the primary	PRIMARY
7	Verify primary is clean and healthy	PRIMARY
8	Wipe standby and rebuild with pg_basebackup	STANDBY
9	Start standby and verify streaming replication	STANDBY + PRIMARY
10	Final health check — both servers	BOTH

STEP 1: Stop the Standby

The standby is broken and wasting resources. Stop it first so it doesn't interfere with cleanup on the primary.

BASH — Stop Standby

```
# On STANDBY:
sudo systemctl stop postgresql-16

# Verify it's stopped:
sudo systemctl status postgresql-16
ps aux | grep postgres
```

STEP 2: Stop the Active Restore on Primary

If the restore is still actively running (pg_restore, psql \i, or any other import process), you need to stop it cleanly.

2.1 — Identify Restore Sessions

SQL — Find Active Restore Sessions

```
-- On PRIMARY: find active restore sessions
SELECT pid, username, datname, now() - xact_start AS duration,
       state, left(query, 80) AS query_snippet
FROM pg_stat_activity
WHERE state = 'active'
      AND query NOT LIKE '%pg_stat_activity%'
ORDER BY duration DESC;
```

2.2 — Cancel the Restore Sessions

Try graceful cancel first. If that doesn't work, use terminate:

SQL — Cancel/Terminate Restore Sessions

```
-- Option A: Graceful cancel (try this first)
-- Replace <pid> with actual PIDs from above query
SELECT pg_cancel_backend(<pid>);

-- Option B: Force terminate (if cancel doesn't work after 30 seconds)
SELECT pg_terminate_backend(<pid>);
```

SQL — Terminate All Sessions on Target Database

```
-- To cancel ALL sessions on a specific database (e.g. 'target_db'):
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'target_db'
      AND pid <> pg_backend_pid();
```

2.3 — If Restore Is Running via External Tool

BASH — Kill External Restore Process

```
# If pg_restore or psql is running in a shell/screen session:
# Find the process:
ps aux | grep -E 'pg_restore|psql'

# Kill it gracefully:
kill <pid>

# If it doesn't stop:
kill -9 <pid>
```

After stopping the restore: Wait 30 seconds and verify no restore sessions remain. The database will contain partially loaded data — that's fine, we'll drop it in the next step.

SQL — Verify Restore Is Stopped

```
-- Verify no restore sessions remain:
SELECT pid, username, datname, state, left(query, 80)
FROM pg_stat_activity
WHERE datname = 'target_db';
-- Should return no active restore queries
```

STEP 3: Drop the Restored Database (Remove Partial Data)

This step removes the partially restored database. Make sure you are dropping the correct database — the one that was the target of the 400 GB restore. Do NOT drop your existing production databases.

3.1 — Identify Which Database to Drop

SQL — List Databases and Sizes

```
-- On PRIMARY: list all databases with their sizes
SELECT datname,
       pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;
```

The partially restored database should be obvious — it will be the large one (200+ GB) that was the restore target.

3.2 — Disconnect All Remaining Sessions from the Target Database

SQL — Disconnect All Sessions from Target DB

```
-- Replace 'target_db' with your actual database name:
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'target_db'
AND pid <> pg_backend_pid();
```

3.3 — Drop the Database

SQL — Drop the Partially Restored Database

```
-- Replace 'target_db' with your actual database name:
DROP DATABASE target_db;

-- If the above fails with 'being accessed by other users':
DROP DATABASE target_db WITH (FORCE); -- PostgreSQL 13+
```

SQL — Verify Database Dropped

```
-- Verify the database is gone:
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;
```

If the restore was into an EXISTING database (not a new one), and you want to restore it to its original state, you'll need to restore from your last backup of that database instead of dropping it. The commands above assume the restore target was a new/separate database.

STEP 4: Drop the Broken Replication Slot

SQL — Check Replication Slot Status

```
-- On PRIMARY: check slot status
SELECT slot_name, slot_type, active, restart_lsn
FROM pg_replication_slots;
```

SQL — Drop Broken Slot ssncpri

```
-- Drop the broken slot:
SELECT pg_drop_replication_slot('ssncpri');

-- Verify it's gone:
SELECT slot_name FROM pg_replication_slots;
-- Should return no rows (or no row for ssncpri)
```

Note: Do NOT create a new slot yet. We will create it automatically during pg_basebackup in Step 8 using the `-S ssncpri -C` flags.

STEP 5: Revert All Parameter Changes on Primary

This removes all ALTER SYSTEM overrides and returns postgresql.conf as the sole configuration source.

5.1 — Check What Overrides Currently Exist

BASH — View Current Overrides

```
# On PRIMARY: view current overrides
cat /apps/pgsql_data/16/postgresql.auto.conf
```

5.2 — Reset Everything

SQL — Reset All Overrides

```
-- On PRIMARY: reset ALL parameter overrides
ALTER SYSTEM RESET ALL;
```

5.3 — Verify the Auto.conf File Is Clean

BASH — Verify Auto.conf Is Clean

```
# Verify postgresql.auto.conf is now empty:
cat /apps/pgsql_data/16/postgresql.auto.conf
# Should be empty or only contain a comment header
```


STEP 6: Restart the Primary

BASH — Restart Primary

```
# On PRIMARY:
sudo systemctl restart postgresql-16

# Verify it started cleanly:
sudo systemctl status postgresql-16
```

STEP 7: Verify Primary Is Clean and Healthy

7.1 — Verify Parameters Are Back to Original

SQL — Verify All Parameters Reverted

```
-- On PRIMARY:
SHOW max_wal_size;           -- Should be back to original (e.g. 1GB)
SHOW min_wal_size;          -- Should be back to original (e.g. 80MB)
SHOW checkpoint_timeout;    -- Should be back to original (e.g. 5min)
SHOW wal_buffers;           -- Should be back to original (e.g. -1)
SHOW work_mem;              -- Should be back to original (e.g. 4MB)
SHOW maintenance_work_mem;  -- Should be back to original (e.g. 64MB)
SHOW wal_keep_size;         -- Should be back to original
SHOW max_slot_wal_keep_size; -- Should be back to original
SHOW synchronous_commit;    -- Should be back to original (e.g. on)
```

7.2 — Verify No Replication Slots Remain

SQL — Verify No Stale Slots

```
SELECT slot_name, active FROM pg_replication_slots;
-- Should return no rows
```

7.3 — Verify the Target Database Is Gone

SQL — Verify Database Cleaned Up

```
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;
-- The partially restored database should NOT appear here
```

7.4 — Check WAL Directory Size Has Reduced

SQL — Run Checkpoint and Check WAL Size

```
-- WAL should start shrinking after checkpoint:
CHECKPOINT;

SELECT pg_size_pretty(sum(size)) AS total_wal
FROM pg_ls_waldir();
```

BASH — Verify Disk Space Recovered

```
# Check disk space recovery:
df -h /apps/pgsql_data/16
df -h /apps/pgsql_data/16/pg_wal
```

7.5 — VACUUM to Reclaim Space (Optional but Recommended)

After dropping a large database, PostgreSQL frees the files immediately. But if you loaded data into an existing database and rolled back, run VACUUM to reclaim space:

SQL — Reclaim Space (Optional)

```
-- Only needed if data was loaded into an existing database:
VACUUM FULL VERBOSE;
```

The primary is now clean and back to its pre-restore state.

STEP 8: Wipe Standby and Rebuild with pg_basebackup

Triple-check you are on the STANDBY server before running any destructive commands. Running these on the primary will destroy your production data.

8.1 — Verify You Are on the Standby Server

BASH — Confirm You Are on Standby

```
# On STANDBY: confirm hostname
hostname

# Confirm PostgreSQL is stopped
sudo systemctl status postgresql-16
ps aux | grep postgres
```

8.2 — Wipe the Standby Data Directory

BASH — Wipe Standby Data Directory

```
# On STANDBY: remove all data
rm -rf /apps/pgsql_data/16/*

# Verify it's empty:
ls -la /apps/pgsql_data/16/
# Should show empty directory
```

8.3 — Run pg_basebackup

BASH — pg_basebackup (Creates Slot ssncri Automatically)

```
# On STANDBY: rebuild from primary
pg_basebackup \
  -h <primary_host_or_ip> \
  -p 5432 \
  -U replication_user \
  -D /apps/pgsql_data/16 \
  -Fp \
  -Xs \
  -P \
  -R \
  -S ssncri \
  -C
```

What each flag does:

- **-h <primary_host_or_ip>** — Replace with your primary's hostname or IP

- **-p 5432** — Port (change if different)
- **-U replication_user** — Replace with your actual replication user
- **-D /apps/pgsql_data/16** — Target data directory
- **-Fp** — Plain format
- **-Xs** — Stream WAL during backup
- **-P** — Show progress
- **-R** — Auto-creates `standby.signal` and sets `primary_conninfo`
- **-S ssncpri -C** — Creates the replication slot `ssncpri` on the primary

8.4 — Fix File Ownership

BASH — Fix Ownership

```
# On STANDBY: ensure postgres owns all files
chown -R postgres:postgres /apps/pgsql_data/16
```

STEP 9: Start Standby and Verify Streaming Replication

BASH — Start Standby

```
# On STANDBY: start PostgreSQL
sudo systemctl start postgresql-16

# Verify it started:
sudo systemctl status postgresql-16
```

9.1 — Verify Standby Is in Recovery Mode

SQL — Verify Standby Status

```
-- On STANDBY:
SELECT pg_is_in_recovery();           -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

9.2 — Verify Replication Slot Is Active on Primary

SQL — Verify Slot ssncpri Active

```
-- On PRIMARY:
SELECT slot_name, slot_type, active, restart_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
-- active must be: t
-- slot_lag should be very small (near 0 bytes)
```

9.3 — Verify Streaming Replication Is Flowing

SQL — Verify Streaming Replication

```
-- On PRIMARY:
SELECT pid, client_addr, state, sent_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must be: streaming
-- replay_lag should be 0 bytes or very small
```

If state = streaming and replay_lag is near zero, replication is fully rebuilt and healthy.

STEP 10: Final Health Check — Both Servers

Run this complete health check to confirm everything is clean:

10.1 — On PRIMARY

SQL — Full Primary Health Check

```
-- Parameters are original values:
SHOW max_wal_size;
SHOW min_wal_size;
SHOW checkpoint_timeout;
SHOW wal_buffers;
SHOW work_mem;
SHOW maintenance_work_mem;
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;

-- postgresql.auto.conf is clean:
-- Run in bash: cat /apps/pgsql_data/16/postgresql.auto.conf

-- Replication slot is healthy:
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
       slot_lag
FROM pg_replication_slots;

-- Streaming replication is working:
SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;

-- Target database is gone (or restored to original):
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;

-- Disk space is recovered:
-- Run in bash: df -h /apps/pgsql_data/16
```

10.2 — On STANDBY

SQL — Full Standby Health Check

```
-- Standby is in recovery:
SELECT pg_is_in_recovery();           -- Must return: t

-- Standby is receiving WAL:
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming

-- Databases match primary (no partial restore data):
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;
```

CLEANUP COMPLETE. Both servers are now back to their pre-restore state. Streaming replication is healthy with slot `ssncpri`. You are ready for next week's restore attempt.

Checklist for Next Week's Restore Attempt

Before starting the restore next week, set these **FIRST** to prevent the same replication breakage:

SQL — Pre-Restore Checklist for Next Week

```
-- Run on BOTH primary and standby BEFORE starting the restore:
ALTER SYSTEM SET wal_keep_size = '100GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';

-- Then restart: STANDBY first, PRIMARY second
-- Monitor replication every 15-30 minutes during restore
-- After restore completes: ALTER SYSTEM RESET ALL; on both servers
```